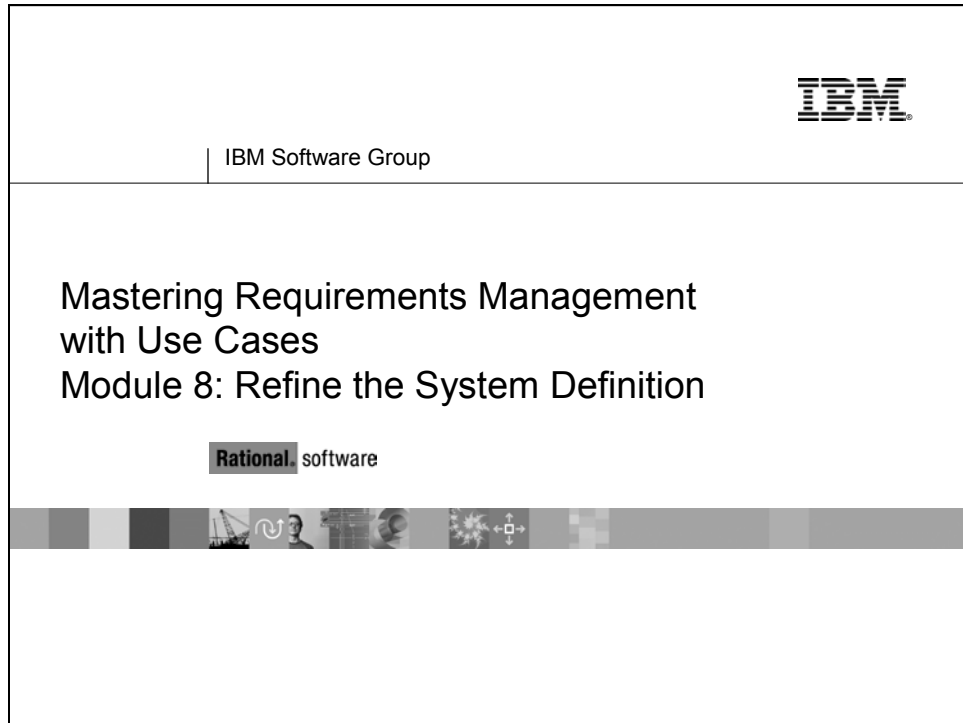


► ► ► **Module 8**

Refine the System Definition



Topics

Features Drive Software Requirements	8-8
How to Specify Functional Requirements?	8-10
What About Requirements NOT in Use Cases?	8-11
Functional Requirements in Declarative Statements	8-12
Detail the Basic Flow of Events	8-18
Structure the Use Case Flows.....	8-20
Detail Alternative Flows.....	8-23
Subflows	8-28
Preconditions	8-33
Postconditions.....	8-34
What About Nonfunctional Requirements?	8-40

Objectives

Objectives

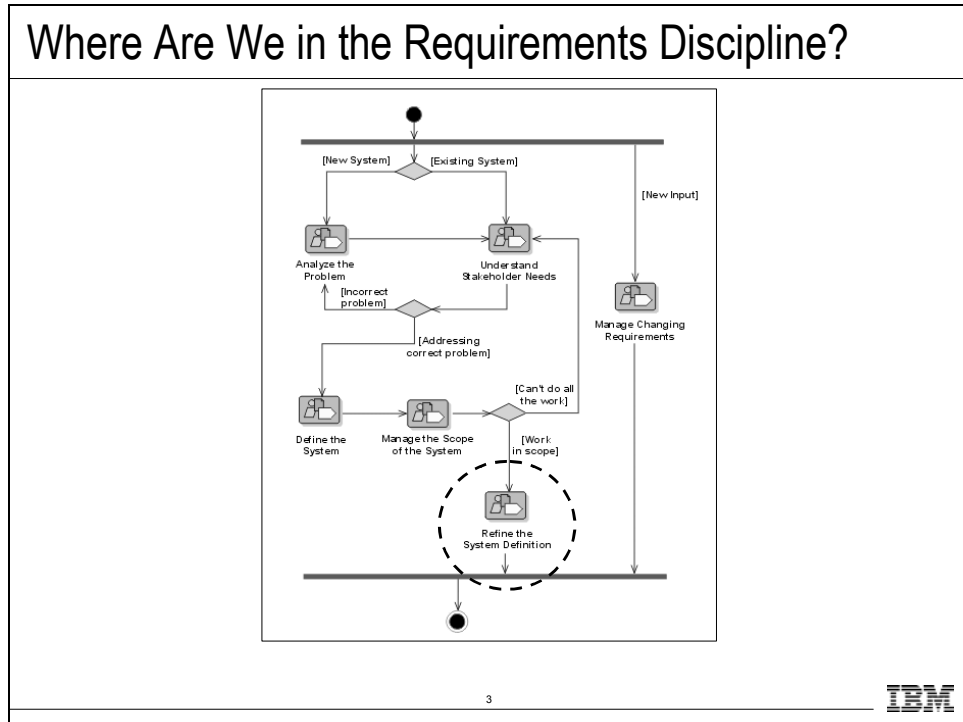
- ♦ Describe design constraints.
- ♦ Identify methods of specifying functional requirements.
- ♦ Describe techniques for writing and structuring use cases.
- ♦ Detail the use case flows.
 - Flows of Events
 - Pre- and postconditions
 - Use case structure
- ♦ Specify nonfunctional requirements.
 - Usability, Reliability, Performance, Supportability

2



Let's move into the heart of the requirements: the software requirements. The goal is to express the required behavior and conditions at a level that implementers and testers can use to drive the further development of the system to be built.

Where Are We in the Requirements Discipline?

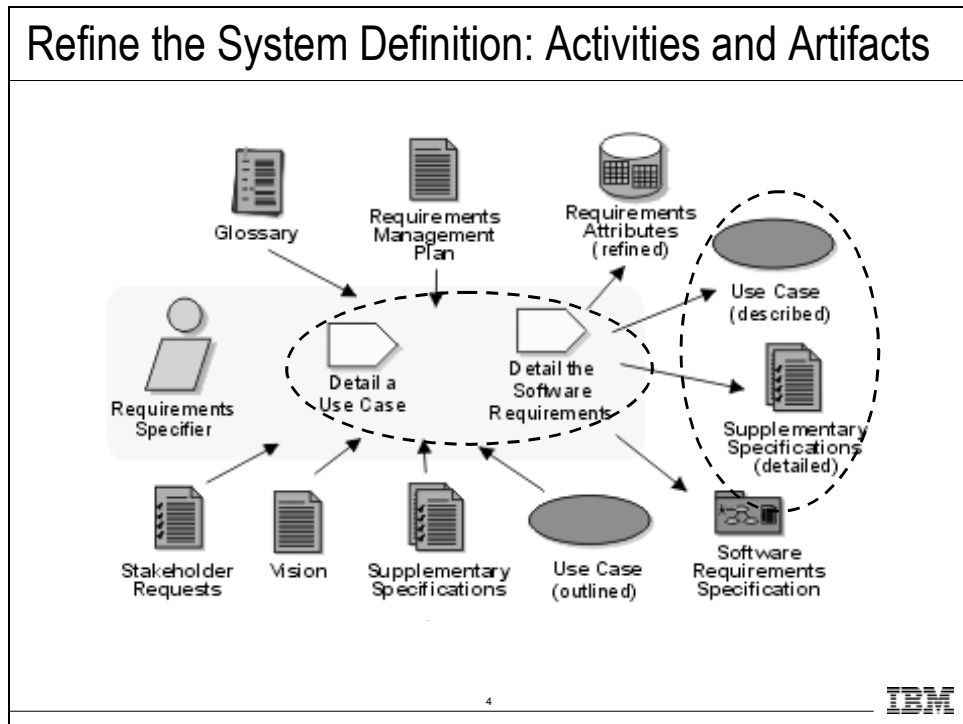


Where does the Refine the System Definition workflow detail fit in our development process?

In the Rational Unified Process®, the workflow detail called *Refine the System Definition* represents activities that you might undertake to further refine the set of requirements for the system.

Where is this done in your own software development process?

Refine the System Definition: Activities and Artifacts



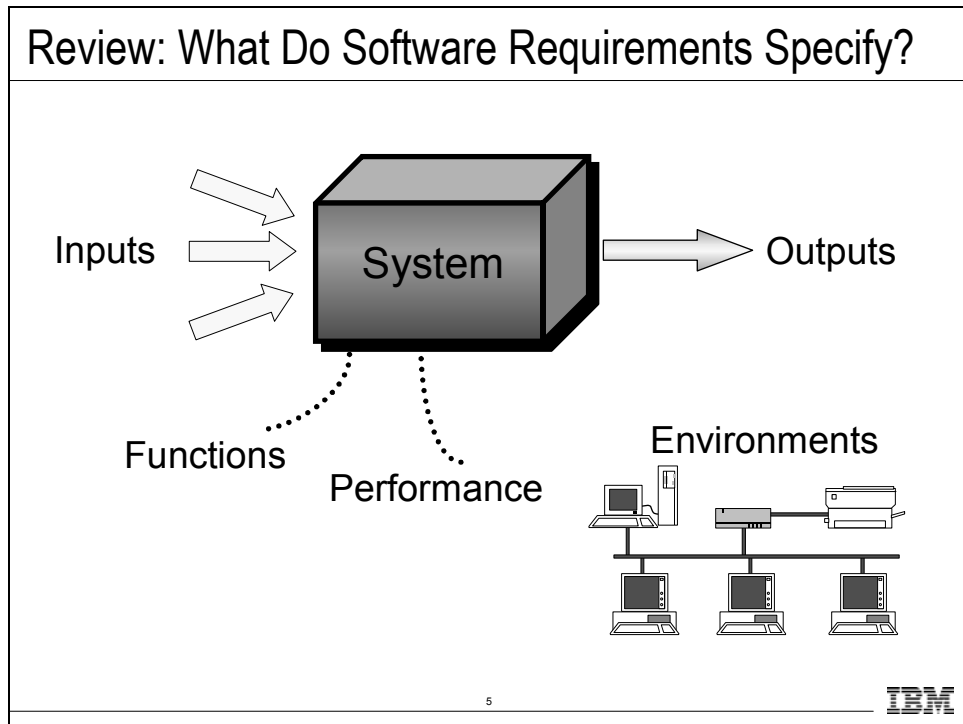
The workflow detail addresses:

- Describing the use case flow of events in detail.
- Detailing Supplementary Specifications.
- Developing a Software Requirements Specification, if more detail is needed.

This workflow detail furthers the understanding of project scope reflected in the set of prioritized product features (often described in the Vision) that it is believed can be achieved by fairly firm budgets and dates. The output is a more in-depth understanding of system functionality expressed in refined, detailed requirements in specification artifacts, and outlined user-interface elements. The specification artifacts can take the form of detailed use cases and Supplementary Specifications, and in some cases a formal Software Requirements Specification may be developed. This work typically starts by reviewing the existing actor definitions and if necessary least briefly describing the actors, then continues with detailing the use cases that have been previously outlined for each actor.

Whenever the requirements specifications are changed, regular reviews and updates to the associated requirements attributes should be done as shown in the Manage Changing Requirements workflow detail.

Review: What Do Software Requirements Specify?



A requirement is a capability or condition to which the system must conform.

Software requirements provide an almost “black-box” definition of the system. They define only the externally observable “What’s” of the system, not the “How’s.”

Of course, there has to be enough of the “How” specified to build the right system, but these should be identified as design constraints.

What About Design Constraints?

What About Design Constraints?

- ♦ A requirement that includes information about the system design or architecture is a design constraint.
 - Distinguish it from other requirements.
 - Place in a special section of your software requirements.
 - Identify the source of each.
 - Document the rationale for each.
- ♦ Examples.
 - An algorithm that is required to be used.
 - A database system that is required to be used.
 - A language that must be used in the implementation.

6



As stated in *Managing Software Requirements: A Unified Approach*, “Requirements should not include information about the system design or architecture. Otherwise, you may accidentally have restricted your team in pursuing whatever design options make the most sense for your application.”

Make sure that you can understand and make clear which requirements are constraints on the design. The purpose of this activity is to try and identify the “project icebergs.” Icebergs are those things that, on the surface, appear much more trivial than they are. Unlike the Titanic, they do not always sink your project, but they may knock it way off course.

You should also make sure you understand the rationale for each design constraint. If the reason for the constraint is someone’s personal preference, then it should be challenged. If, however, the constraint has been imposed by the customer or legislature and non-adherence to the constraint affects your ability to be paid (or you could be sued), then it should be noted as a constraint and stored in the *Constraints* section of the Vision. By separating the constraints, you can help ensure that they are distinguished from other requirements.

An example of a design constraint:

Constraint: The middle tier of the system must be developed in Java.

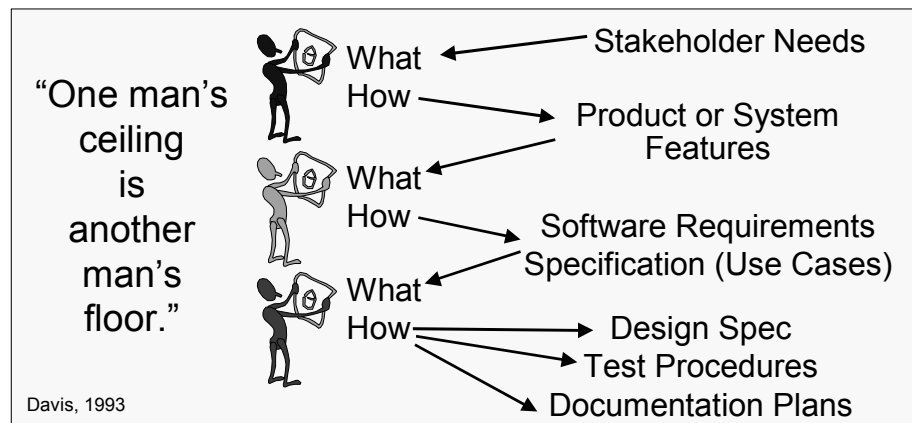
Rationale: A previous implementation provides a set of wrappers for the existing mainframe infrastructure that was developed in Java. Java must be used in order to leverage the existing investment.

The What-Versus-How Dilemma

The What-Versus-How Dilemma

Question: How can you tell a requirement from design?

Answer: It depends on your point of view.



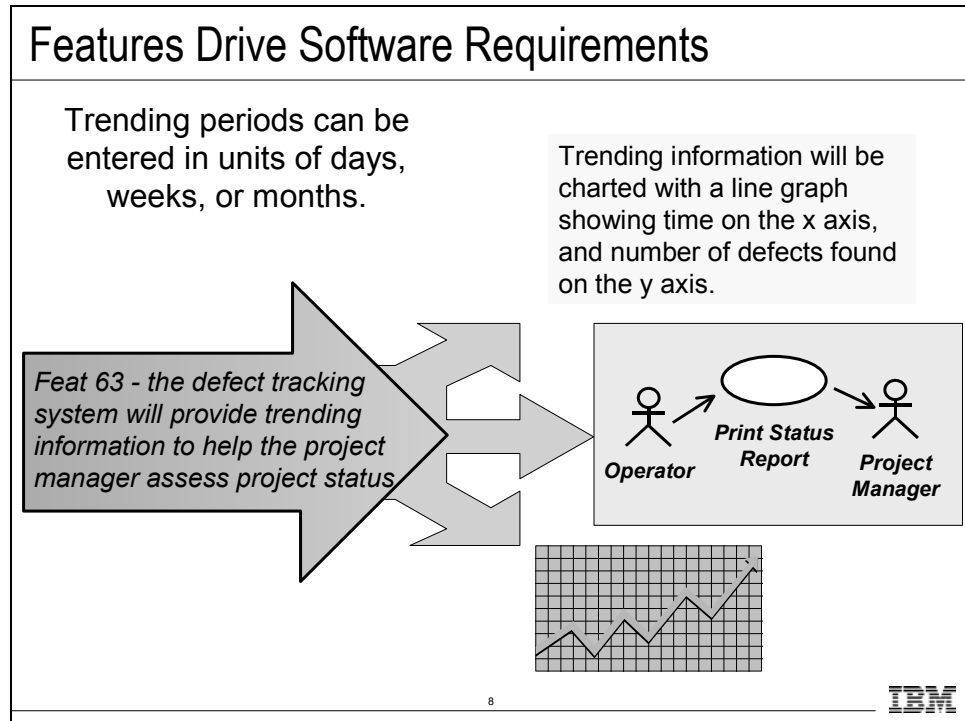
A question that frequently comes up when writing requirements is, "How can I write a requirement that tells the audience 'what' to do without specifying 'how' to do it?"

Unfortunately, there is no one right answer. It depends on the intended point of view of the particular document reader.

Example

- When writing the Vision document, the "what" are the user needs, and the "how" are the features.
- For the Use-Case Specifier, the "what" are the features, and the "how" are the software requirements, use case specification or supplementary requirements.
- For a Designer, the "what" is the use-case specification, the "how" is the design model.
- For the Coder the "what" is the design model, the "how" is the code.

Features Drive Software Requirements



The features that you identified for the system (see Module 6) drive the definition of the software requirements.

The software requirements may be specified in many forms, including use cases and declarative sentences.

There is no clear mapping between features and use cases. It could be 1:1, 1:n or n:1.

A Closer Look at the “F” of FURPS

A Closer Look at the “F” of FURPS		
Functionality	Feature set capabilities, security, generality	
Usability	Human factors aesthetics, consistency, documentation	
Reliability	Frequency/severity of failure, recoverability, predictability, accuracy, MTBF	
Performance	Speed efficiency, resource usage, throughput, response time	
Supportability	Testability Adaptability Compatibility Serviceability Localizability	Extensibility Maintainability Configurability Installability Robustness

Grady, 1992

9



FURPS provides a checklist of items to make sure you are building a quality system.

The FURPS acronym represents a portion of the types of requirements for which you should be searching. It reminds you of the types of nonfunctional requirements you should look for.

Let's look at the **F**unctionality category in more depth. How can you specify functional requirements?

How to Specify Functional Requirements?

How to Specify Functional Requirements?

- ♦ Use both use cases and declarative statements.
 - Both are necessary to understand any system of significant complexity.
- ♦ Give preference to use cases, where appropriate.



Use Cases

+

The system shall ...
The system must ...
The system shall ...

Declarative Statements

? Which one to choose ?

10



There is no one right answer to the question of how to write requirements about system functionality. For many projects, the answer is a mix: some functional requirements are expressed in use cases, and others are expressed in declarative statements.

How does the analyst decide when to use use cases and when to use declarative statements of requirements? The mix varies greatly from organization to organization and from project to project.

When deciding whether particular behavior is best modeled with use cases, consider:

- Context of the application and the project.
- Skills and comfort level of the developer organization.
- Client preferences and environment.
- Risks in your project and how best to address those risks.

What About Requirements NOT in Use Cases?

What About Requirements NOT in Use Cases?

- ♦ Write a declarative statement to describe the software requirement.
 - Use a keyword to aid identification, for example “shall.”
 - Number and title each requirement.
 - Group related requirements.
- ♦ Use language the team easily understands.
 - Use simple sentence structure.
 - Subject + active verb.
 - Be concise and thorough.
 - State the requirement in 1 to 3 sentences.
 - Make the requirement complete.
 - Use terms from the glossary.

11



Use cases may contain many of the detailed functional software requirements. But many other detailed software requirements (both functional and nonfunctional) are best stated in declarative sentences that are independent of use cases.

Note: subject and active verb does not necessarily apply to non-English languages. The point is to strive for a name that clearly, concisely and unambiguously describes what the actor intends to use the system for.

When specifying software requirements as declarative statements, keep each requirement short and easy to understand. Number and title each requirement so you can refer to it easily, especially when you are reviewing it.

Requirements presented in a context are easier to understand. Grouping related requirements together provides some context. For example, all requirements about reliability may be grouped together. Both hierarchical relationships and packages can be used to group declarative statements of software requirements.

Remember that the requirement specifications are for the users to read.

Requirements have to be written for the user. Do not alienate them by using language they do not understand. Use simple sentences, with a straightforward sentence structure: subject followed by active verb and object. Use terms from your glossary.

The advantage of using a keyword, such as “shall,” to identify each requirement is that it is immediately clear which statements are requirements and which are merely additional text. The disadvantage is that the use of a keyword in some requirements may sometimes make them stilted or more difficult to understand.

Functional Requirements in Declarative Statements

Functional Requirements in Declarative Statements

- ♦ Some functional requirements are easier to state declaratively.

- Example: RU e-st System

1. Localization:

SUPL120: The RU e-st system screens shall support switching to another language for all messages and menus in the user's preferred language at any time. The languages supported shall include English, Spanish, French, German, and Swedish.

12



Some functional software requirements are difficult to describe within the context of a use case. These functional requirements can be stated declaratively in the Supplementary Specifications. For example, localization requires that messages to the user are displayed in the user's preferred language. In each step of each use case that displays a message, you could say something like, "The system prompts the customer, in their preferred language, to enter their personal details." As well, you can make a single declarative statement that applies to all displays in the entire system.

For example, when describing the requirements for a language compiler, you will find that there are many requirements for *compiler switches* that are better described declaratively. One such switch could be to tell the compiler to select between optimizing compilation for an Intel® Pentium® II or an Intel Pentium III. Because there is not interaction with the user for these switches, these requirements are better specified declaratively.

Where Are Declarative Statements Specified?

Where Are Declarative Statements Specified?

- ◆ Does the requirement apply to a use case?
 - Specify in the Use-Case Specification
 - In “Special Requirements” property
- ◆ Does the requirement apply to the whole system?
 - Specify in the “Supplementary Specifications”



RUCS8: Supplementary Specification

13

IBM

If a declarative statement applies to a particular use case, then it is usually specified in the “Special Requirements” property of the use-case specification for the particular use case to which it applies.

For example, for the Get Quote use case in an RU e-st system, there may be a nonfunctional requirement about the maximum allowable time for the system to respond to a request to get a quote. This performance requirement may be placed in the “Special Requirements” property of the Get Quote use case.

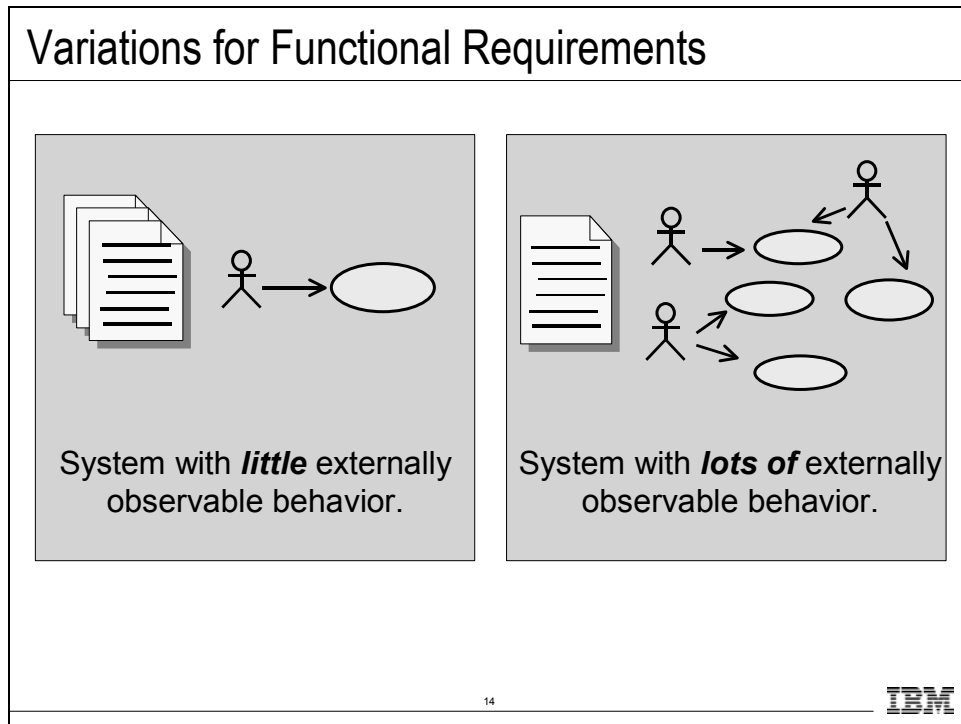
Supplementary Specifications contain the declarative statements that are not about a particular use case. For example, for the RU e-st system, the Supplementary Specifications may contain a nonfunctional requirement that the system must be able to run with either MS Internet Explorer or Netscape Navigator.

Does the requirement apply to multiple use cases?

- Replicate the requirement in each use case, or put it in the Supplementary Specification.
- The decision as to the best location is based on conjecture and guided by experience.

Tip: If you are using a tool like RequisitePro®, a useful technique for storing supplementary requirements is to locate them in the database and then use SoDA® to create a Supplementary Specification for you.

Variations for Functional Requirements



How much of the functional requirements do you specify with use cases and how much in declarative statements? It varies from project to project. Some projects have many use cases, and only have a few declarative statements. In other projects, it is just the opposite. It depends on the amount of externally observable behavior in the system.

Our preference is to model functional requirements with use cases, because of the advantages mentioned throughout the course. For example, in the RU e-st system, the Get Quote use case shows the whole sequence of behavior to get a price quote. Each system behavior is a functional requirement. The sequence gives context to the functional requirements. But not all behavior is best modeled with use cases. For example, you could describe functional requirements for a Java compiler with use cases, but standard Backus Naur Form (BNF) would be more appropriate and concise.

Another example, suppose you are adding localization functionality to an existing product so that messages display in the language of the user's choice. You could change existing use cases to describe the multi-language display of messages, but you might prefer to write a few declarative functional requirements instead.

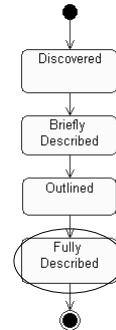
Some clients insist on the traditional approach of specifying all requirements in declarative statements. You must develop the traditional SRS, but you can also make use cases for your developers. You may want to maintain all requirements in two different formats (declarative and use cases), and then trace between them to help keep them consistent. The disadvantage is that you have double work to keep two copies of the requirements. Instead, you may develop only a few illustrative use cases to show central functionality and architecturally significant use cases.

Nonfunctional requirements, such as performance requirements, are almost always specified with declarative statements. Specifying nonfunctional requirements is covered later in this module.

Detail Each Use Case

Detail Each Use Case

1. Identify actors and use cases.
 - Brief Description
2. Outline each use case.
 - Basic Flow of Events
 - Alternative Flows of Events
3. Detail each use case. ✓
 - Detail the flow of events.
 - Structure each use case flows of events.
 - Add detail.
 - Pre- and postconditions, special requirements, relationships, use-case diagrams, and so on.



Mastering Requirements Management with Use Cases
Copyright © 1998-2003 Rational Software, all rights reserved

15

Rational
the software development company

There are a number of brief iterations that your use cases go through during development. At this point in development, you are concerned with detailing our use case's flows of events. Add detail to the outlines already written. While in this activity, pay attention to the structure of the flows to make sure that they make the use case as easy to understand as possible.

Generally, you do not detail an entire use case at a time. In the *Manage the Scope of the System* workflow, you prioritized a list of scenarios that should be detailed in the next iteration. You should detail, implement, and test your use cases according to this list.

Why Detail Use Cases?

Why Detail Use Cases?

- ◆ Specify the software requirements.
 - Create a specification that can be implemented.
- ◆ Clarify important details in flow of events.
 - What the actor does.
 - What the system does in response.
 - What information is exchanged.
- ◆ Describe additional information.
 - Preconditions
 - Postconditions

16

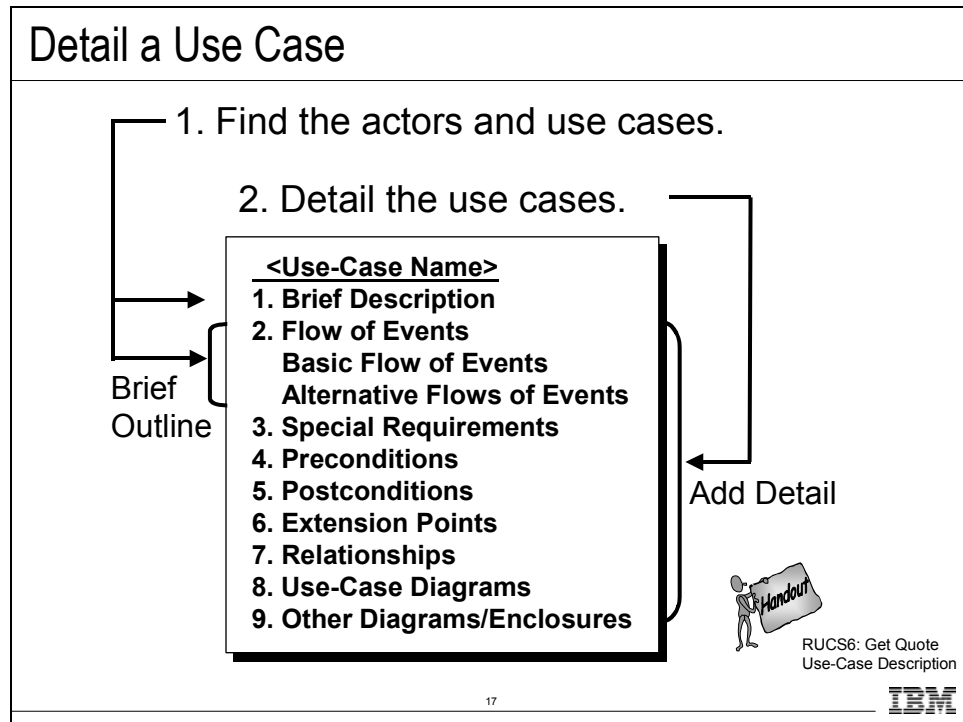


Detailing the use-case description is where the “real work” in use-case modeling occurs. The outline of the flow of events in each use case that you developed earlier lacks important details. The main purpose of detailing a use case is to elaborate the step-by-step outline of the flow of events. In addition, specify other properties of a use case, as shown on the template on the previous slide.

Without a detailed description of the flows of events, it is nearly impossible to implement a solution. The reason is that the use-case outline did not contain any real software requirements. Instead, the outline just contained a high-level description of what the system does.

The detailed flow of events includes what the actor does and what the system does in response. The actions that the system does are the software requirements: they are what the user wants the system to do.

Detail a Use Case



This is a template for a Use-Case Specification.

In the “Find Actors and Use Cases” activity, you identified the use cases and sketched a step-by-step outline for each use case. Now you are ready to begin the “Detail a Use Case” activity.

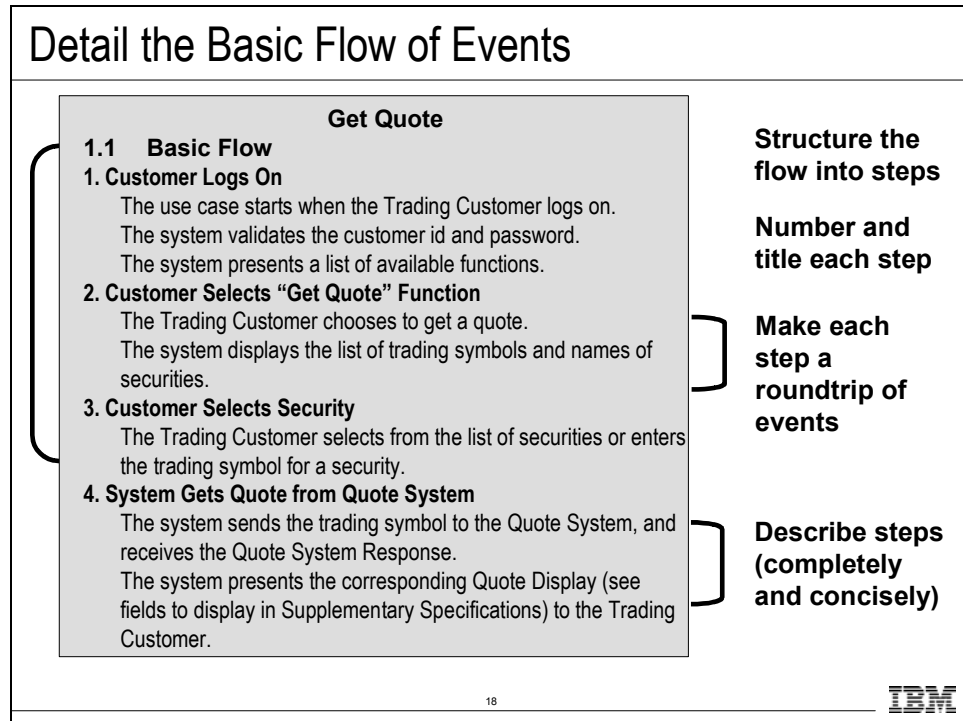
For each use case, start with the step-by-step outline and gradually add detail to each step. You want to understand what really happens so you don't miss anything. Typically, you begin with refining the Basic Flow and then refine the Alternative Flows.

Work together with users to revise and detail the outline. Expect to revise the outline and the details many times.

There is no minimum or maximum size for a use-case specification. It needs to be big enough to contain an unambiguous and concise description of how the system behaves. Be sure your description is complete and that you have answered all the questions that people may have about the events in the use case.

The Use-Case Specification describes the details of a particular use case. In addition to the Flow of Events, there are other properties that detail a use case. Rational recommends that you use a standard template for your use-cases, such as the one shown here (from the Rational Unified Process®). Whether or not you have any properties within the section, stick to the standard outline.

Detail the Basic Flow of Events



There are many styles for writing flows of events. One of the styles Rational® uses is to number and title each step. This gives the reader an overview of the flow without all the details and can refer to a step when it is being reviewed.

There are many other styles. You can encourage consistency in style by specifying acceptable style guidelines when you write your Use-Case Modeling Guidelines. When you detail each step in the outline, be sure to describe the flow of events, not only what the system is doing. A suggestion for how to enforce this is to start every step with "When the [actor] ..."

Rational recommends making each step in the flow of events show a roundtrip of events, usually in the form of messages:

- What the actor does: <Actor> messages <System> , and
- What the system does in response: <System> messages <An Actor> ...

Making each step a roundtrip ensures testability, ensures adherence to the purpose of a use case, and makes sequence diagrams far easier to develop and read. However, observe that step 3 is not a roundtrip of events. In this particular case, it made the use case description easier to read if we described the user action on its own. You should strive for roundtrip descriptions, but they are not always be practical.

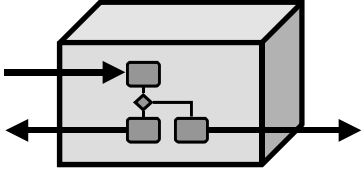
Some interactions are system-controlled. That is, after the user's initial request to begin the use case, the system controls the interaction: the system asks for information and the user supplies information. The system asks for additional information and the user supplies it, and so on. In these cases, a roundtrip within a step would begin with system actions and then have the user response.

Manage the Detail

Manage the Detail



Black Box



White Box

- ◆ Know your audience.
- ◆ Strive for black box.
- ◆ Some white box text may improve understandability because it makes the use case more concrete.

19


A black-box use case contains no description of the system's internal processing. A white-box use case contains descriptions of the system's internal processing.

The problem with black-box use cases is that sometimes the description becomes too trivial to be of use. This is particularly true in systems where there is little externally observable behavior.

The problem with white-box use cases is that by describing the internal processing you can accidentally influence the design of the system by stating what the system does inside and when it does it. Sometimes it is useful to provide a little internal description because it gives some context to the next interaction with an actor.

The question that has plagued use-case authors for many years is: "Just how much detail is enough when writing a use case?"

The answer is that you should adapt the amount of detail to suit your audience. A user is typically interested in a black-box description of the system. A subject-matter expert would be interested in much more information. Often there are important details that the designer should not be left to "figure out," rather a detailed knowledge of the domain is required to describe what the system does.

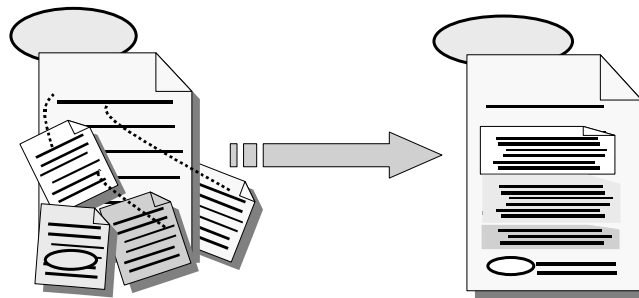
This is an example of where tool support can help you manage the detail. You can use the features of your word processor to turn on or turn off the detail based on your audience. An example of this is the "hidden text" feature in Microsoft® Word. Internal descriptions are written in a style that uses hidden text that can be suppressed later.

When writing your use cases, you should strive as much as possible for black-box use cases. If you do need to add internal processing information, you should do it with caution and attention to your audience.

Structure the Use Case Flows

Structure the Use Case Flows

- ♦ Internal organization of the use case.
 - Increase readability.
 - Make the requirements more understandable.
- ♦ Document acceptable styles in the Use-Case-Modeling Guidelines.



RUCS11: Use-Case
Modeling Guidelines



20

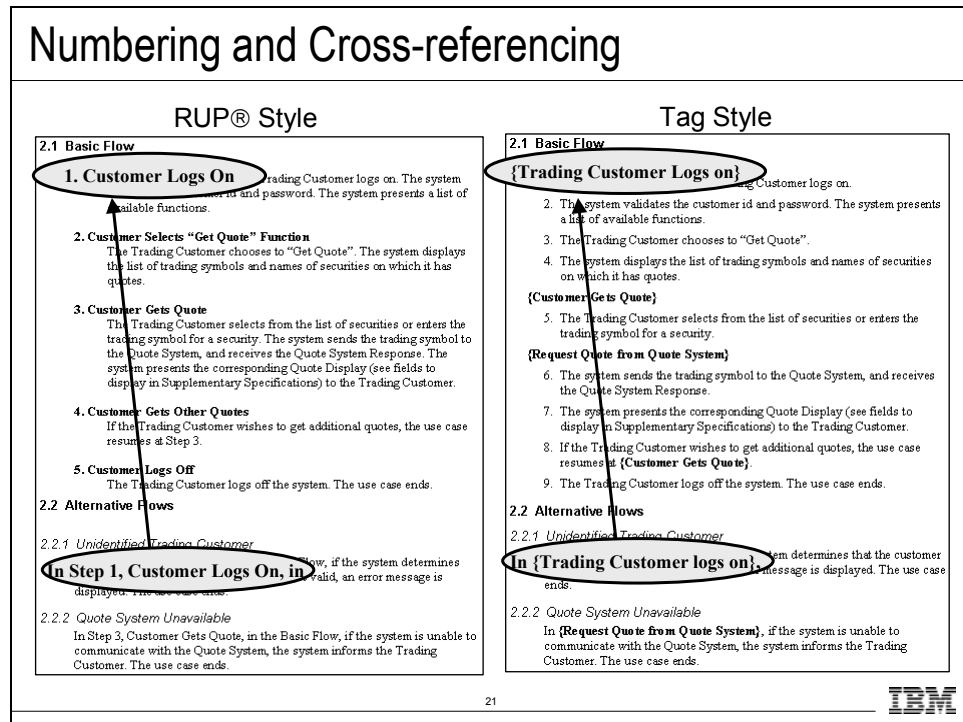
Writing a use case description should not be a “free for all” approach. For the same reasons organizations have coding standards to promote understanding and productivity, it is important to ensure consistency of the use case descriptions with use-case modeling guidelines.

The structure of your use cases is one of the most important and overlooked features of your requirements set. Often teams focus on structuring the model itself (this is covered in module ten) instead of focusing on the internal organization of the information within each specification.

Your use-case modeling guidelines should describe acceptable use case styles. Things to consider standardizing are:

- General format of a flow (paragraphing).
- Labeling and numbering conventions.
- How do you repeat behavior?
- How do you describe optional behavior?
- How do you describe data in forms?
- How do you map to business rules?
- How do you reuse behavior?
- Whether descriptions of internal behavior are allowed.
- Which features of your Word processor will you use to maintain internal cross-references?
- Model structuring guidelines («include», «extend» and generalization).

Numbering and Cross-referencing



There are many styles used for writing use cases. The style you choose should take into consideration the capabilities of your authoring tool, promote clarity, and ease the authoring and maintenance process.

Use cases typically have a great deal of cross-referenced information within them. The overhead of maintaining this cross referencing is what typically causes use cases to become out of date. In both of the above cases, cross-referencing can ease the authoring process by ensuring that the correct labels are used and updated. If you change the text or location of the label, then the location that references the label is also updated.

Word processors, such as Microsoft® Word and Adobe® FrameMaker®, have powerful book marking and cross-referencing capabilities that can ease the maintenance of cross references.

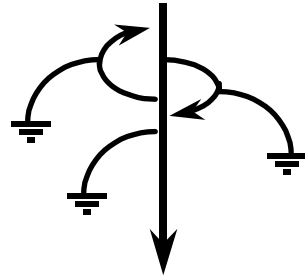
The RUP® style use case shown above has the advantage that an overview of the flow of events can be obtained by reading the headings. If you want the detail you look at the subflow under each heading. The disadvantage of this approach is that internal cross references can be hard to maintain automatically if you insert a heading (high-level step).

The Tag style does not have the overview capability, but is easier to maintain because referenced locations are marked with a label. Inserting steps has no effect on the location of the label. The tag is identified visually by enclosing the text in braces. The tag can be cross referenced by name from other locations in the use-case specification. You only need to create a label for locations that need to be referenced from some other part of the use case. This style is typically easy and fast to author and requires little configuration of the document template.

Review: Flows of Events (Basic and Alternative)

Review: Flows of Events (Basic and Alternative)

- ♦ One Basic Flow
 - Happy day scenario
 - Successful scenario from start to finish
- ♦ Many Alternative Flows
 - Regular variants
 - Odd cases
 - Exceptional (error) flows



Flow: A sequential set of steps.

Detail Alternative Flows

Detail Alternative Flows	
Alternative Flows A3 Request Additional Quotes <u>In Step 3, Customer Gets Quote, in the Basic Flow, if the customer wants additional quotes.</u> <u>The use case continues at Step 3.</u> A4 Quit If at any time the Trading Customer decides to quit. The use case ends. A5 Unknown Trading Symbol <u>In Step 3, Customer Gets Quote, in the Basic Flow, if the system cannot recognize the trading symbol, the system notifies the Trading Customer that the trading symbol is not recognizable.</u> <u>The use case continues at the beginning of Step 3.</u>	Describe what happens Location Condition Actions Resume location

IBM

There are always different ways to structure the text. The main goal is to make the document easy to read.

Describe the alternative flows in their own section, not within the basic flow (although some have found it useful to add pointers where these may be inserted for clarification purposes).

This slide shows both specific and generic alternative flows from the RU e-st example. A specific alternative flow occurs at a specific step in the basic (or other) flow. A generic alternative flow can occur anywhere in another flow.

For specific alternative flows, the following information is detailed:

- The start location in the basic or another subflow where the alternative flow is triggered.
Example, "In Step 3 of the Basic Flow."
- The condition that triggers its start .
Example, "If the line goes down."
- The actions taken in the alternative flow.
- Where the basic or another subflow is resumed after the alternative flow ends.
Example, "Resume at Step 7 of the Basic Flow."
If the use case ends in the alternative flow, state that "The use case ends." in the alternative flow.

For generic alternative flows, there is no start location because a generic alternative flow can begin anywhere.

Avoid Inline Conditional Behavior

Avoid Inline Conditional Behavior

- ♦ Make identifying scenarios difficult.
- ♦ Harder to test and implement.

Preference
should be for
alternative flows.

2.1 Basic Flow

1. Customer Logs On

...

2. Customer Selects "Get Quote" Function

...

3. Customer Gets Quote

The Trading Customer selects from the list of securities or enters the trading symbol for a security. The system sends the trading symbol to the Quote System, and receives the Quote System Response.

IF the system is unable to communicate with the Quote System THEN

The system informs the Trading Customer

The use case ends.

ELSE

The system presents the corresponding Quote Display (see fields to display in Supplementary Specifications) to the Trading Customer.

ENDIF

4. Customer Gets Other Quotes

...

24

IBM

It is common to see conditional statements such as IF-THEN-ELSE-ENDIF in use-case descriptions. The primary reason for including conditional statements in a use-case flow is that they are very natural and convenient to write. However, every time you include a conditional statement you add a level of complexity to the use case. This is because the reader needs to keep in mind the condition that got them to the subflow. With one level of condition this is easy, but as you add more conditions, the level of complexity goes up an order of magnitude.

For example:

```
IF a THEN
  do something
  IF b THEN
    do something else
  ELSE
    do something else again
  ENDIF
ENDIF
```

Implementing and writing test scripts for use cases that make excessive use of conditional statements is difficult and prone to error. A far better approach is to use alternative flows.

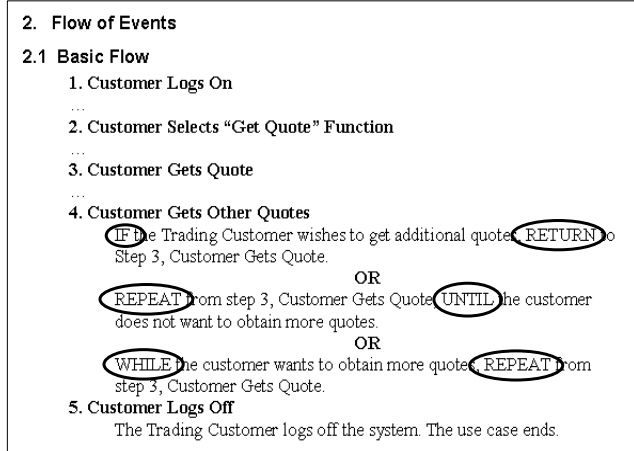
There are tradeoffs. A use case with many alternative flows becomes difficult to read because the flows are fragmented throughout the document. The side effects of over use of conditional statements far outweighs this problem.

Avoid Inline Repetitive Behavior

Avoid Inline Repetitive Behavior

- ♦ Make identifying scenarios difficult.
- ♦ Harder to test and implement.

Preference
should be for
alternative flows.



25

IBM

Repeating behavior is a common requirement when writing use-case descriptions. The example above provides three common styles. Notice that the keywords are in upper case. This style is used to make control points of the description stand out from the rest of the text. This is useful for testers when writing test scripts.

The common structure in all of the examples is that there is a condition that is tested and a location to branch to if the condition is true.

The problem with using "looping constructs" in the use-case description is that it is harder to identify the alternative path. This means that testers and implementers can more readily make mistakes.

Another approach to the three examples above is to use alternative flows to repeat the behavior. In that case, you would have an alternative flow called "**Customer Wants More Quotes**" and have it resume at Step 3.

Choose one style for all use cases in your project and document that style in the use-case modeling guidelines. This ensures consistency and promotes readability of your use cases.

Alternative Flows Versus Inline Conditional Behavior

Alternative Flows Versus Inline Conditional Behavior

- ♦ Alternative Flows
 - Pros
 - Can be used anywhere there is conditional behavior.
 - ♦ REPEAT-UNTIL, IF-THEN-ELSE-ENDIF
 - Make identifying scenarios simpler.
 - ♦ Helps testing and implementation
 - Cons
 - Increased complexity in maintaining cross-references.
- ♦ Inline Conditional Behavior
 - Pros
 - Easier to handle small variations in flows.
 - Cons
 - Harder to identify scenarios, test and implement.

26



It has been reported that 60-80 percent of software is written to cater for error and abnormal conditions. This figure is similar when writing use cases.

In a use case, the scenario (when everything goes right) is described in the Basic Flow. Everything else that happens (errors, exceptions, regular variations) are described as alternatives to the basic flow.

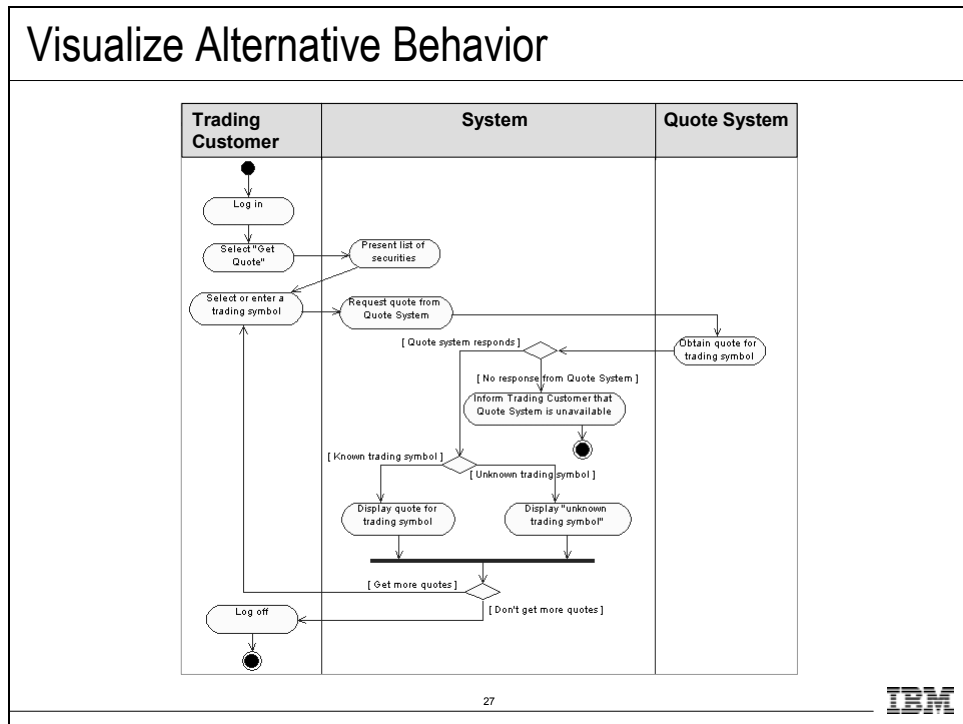
If you included all the alternatives in the basic flow, you would have such a vast number of condition statements and loops that the requirements would start to look like code and be impossible to understand, implement, and test.

Alternative flows are used to describe the variations from the “happy day” scenario (when everything goes right). They have the benefit of making each flow easier to understand, implement, and test in isolation. Unfortunately alternative flows have the disadvantage of making the use-case description fragmented and harder to maintain the internal references. This is a small price to pay when you consider the benefits of making your requirements easier to understand.

Alternative flows can be used in place of looping constructs of conditional statements, thereby reducing the complexity of the flows.

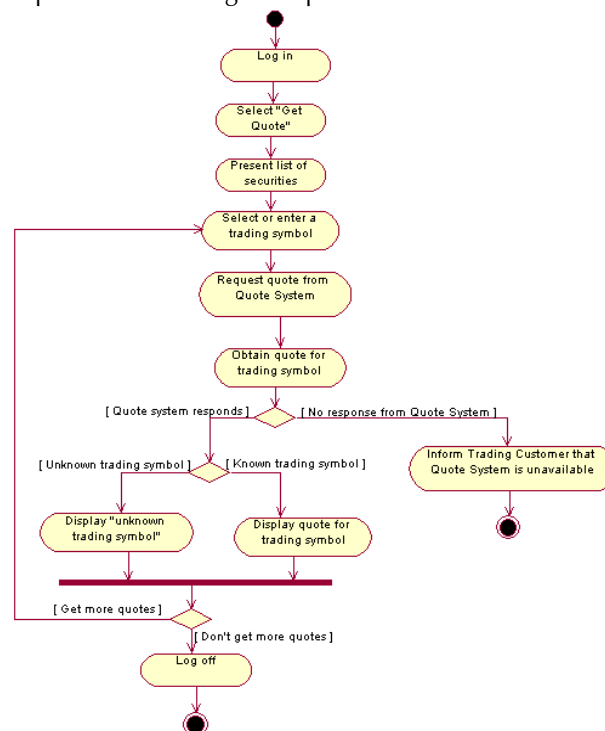
When you use alternative flows, the importance of identifying your scenarios becomes even more apparent. When you implement a use case, you do it one or more scenarios at a time. Ensure that all your alternative flows are included in at least one scenario so that it is implemented and tested.

Visualize Alternative Behavior



Activity diagrams can be used to visualize alternative flows.

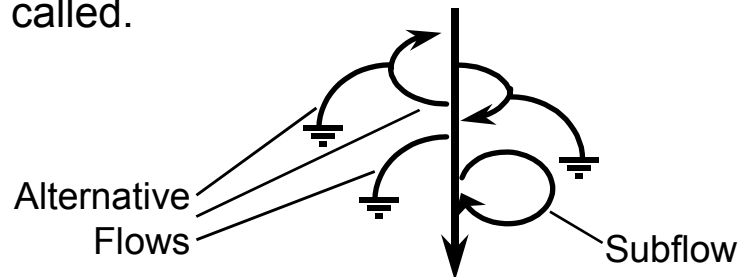
It is important that you make use of swim lanes. If you do not, then the activities float in a “semantic emptiness” because they cannot be anchored as a responsibility of any one or thing. Compare the following example with the one above.



Subflows

Subflows

- ♦ Increase clarity.
- ♦ Allow internal reuse of requirements.
- ♦ Unlike an alternative flow, are explicitly called.
- ♦ Always return to the line after they were called.



28

IBM

When a particular flow becomes overly complex, a common practice is to factor out the complex flow and place it in an included use case (Covered in module 10). While the author had the best intentions of reducing the complexity of the flow, he/she functionally decomposed the system and made it harder to understand the requirements of the system. The reader now has to look in two documents to understand the requirements.

An alternative approach is to use subflows (or *named subflow* in the book *Use Case Modeling* by Kurt Bittner and Ian Spence). A subflow can be thought of as an “internal include.”

Benefits to this approach:

- The flow is still contained in the same use-case specification.
- The flow can be used to increase clarity by factoring out complex flows.
- The flow allows the reuse of flows within the same use case. This is because subflows can be called from multiple places.

The difference between an alternative flow and a subflow is that alternative flows insert themselves into another flow. The flow it inserts itself into has no knowledge of the alternative flow. An alternative flow may also resume at any place within the use case. A subflow is explicitly called from a flow. When a subflow complete it always returns to the line after it was called from. It is similar in concept to a subroutine call in some programming languages.

Subflows are not listed as part of a scenario. By definition, if the calling flow is in the scenario, the subflow is also in the scenario.

As with any other flow, a subflow may have alternative flows.

Example Subflow

Example Subflow

Subflows

1. **Use Case Name: Register for Courses**
- 1.1 **Brief Description**
...
2. **Flow of Events**
- 2.1 **Basic Flow**
 1. Log On
...
 2. Select 'Create a Schedule'
...
 3. **Obtain Course Information**
Perform subflow S1: Obtain Course Information
 4. **Select Courses**
...
 5. Submit Schedule
...
 6. Accept Completed Schedule
...
- 2.2 **Subflows**
 - 2.2.1 **S1: Obtain Course Information**
The student requests a list of course offerings. The student can search the list by department, professor or topic to obtain desired course information. The system retrieves a list of available course offerings from the Course Catalog System and displays the list to the student.



RUCS6.2: Execute Trade UC Spec

29


This is an example of how to write a subflow. Notice that subflows have a section of their own in the use case specification. In this style, the subflows have an identifier in front of them (S1, S2 , and so on) to help identify them as subflows.

Unlike alternative flows, subflows do not need a line that says where they return to because they always return to the line after they were called.

In the example, once the subflow **S1: Obtain Course Information** completes, the flow will resume at **4. Select Course**.

The Execute Trade Use-Case Specification (RUCS6.2) in the Student Workbook has a complete example of a subflow that is referenced from multiple locations in the document.

Exercise 8.1: Choose a Style

Exercise 8.1: Choose a Style

- ♦ Review different flow of events descriptions.
- ♦ Identify the intended audience of each.
- ♦ Identify the easiest to read and understand.
- ♦ Compare and contrast the different styles.



30

IBM

See Student workbook Exercise 8.1: Choose a Style.

Leave the User Interface Out of the Use Case

Leave the User Interface Out of the Use Case

- ♦ Text is not good at describing visual things.
- ♦ Use cases are user-interface agnostic.
- ♦ Describe user Interfaces during Analysis and Design with:
 - User-experience model or prototypes

Words to **AVOID**

Click	Drag	Form
Open	Close	Drop
Button	Field	Drop-down
Pop-up	Scroll	Browse
Record	Window	

Words to Use

Prompts	Chooses
Initiates	Specifies
Submits	Selects
Starts	Displays
Informs	



RUCS11: Use-Case
Modeling Guidelines

31

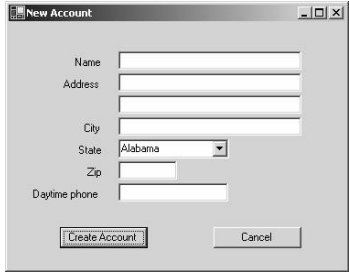


There is truth in the saying “a picture is worth a thousand words.” Use cases were never intended to describe a user interface. Describing a visual thing textually is always open to interpretation and very hard to review. For example, picture a bucket. You may imagine that it is a plastic container with an open top and it has a handle. But someone else may imagine it as being made of metal, has a handle, is painted red, and has the word “Fire” written on it. Everyone interprets descriptions differently. For the same reason, user interfaces are best described visually.

Another reason to keep a user interface out of a use-case description is that the requirements of the system may be the same regardless of the technology used to interact with its surroundings. For example, we could implement the RU e-st system on a 80x24 character text terminal or on a system running Microsoft® Windows® 2000. The use cases should not be different, regardless of the implementation.

Consider the use-case description for a mobile phone. One of the use cases may be “Receive Call.” In this use case, the system needs to notify the user that there is an incoming call. Typically use-case authors would write “The system rings the phone,” but with modern phones the ring tone could be turned off. In fact, the phone could ring, vibrate, display a message on the screen or any combination of the three, depending on which features have been enabled. In this case, the use-case description should be “The system notifies the user of an incoming call. Refer to the special requirements for a description of how the system can notify the user.” In the special requirements, there would be a matrix of how the system can notify the user versus the features that have been enabled in the system. This tells the implementer and tester that this is configurable and the notification method is based on the system’s configuration.

Use the Glossary Effectively

Use the Glossary Effectively	
Use Case	Implementation
5. Enter Customer Info The system prompts the Customer to enter their <i>Customer Details</i>. The Customer enters the <i>Customer Details</i>. The Customer creates the account.	
	Glossary Customer Details: Information that uniquely identifies and provides contact information for a customer located in the U.S.A. The information consists of: Name, two address lines, city, state, zip code, and daytime phone number.

32



Describing how actors interact with forms is common in use cases. There are many techniques authors use, such as placing them in the flow or in the special requirements.

For example:

5. Enter Customer Info

The system prompts the Customer to enter their Name, Address (two lines), City, State, Zip code, and Phone number.

The Customer enters the information.

The Customer creates the account.

This style has a tendency to distract the reader from the real requirements and is harder to maintain.

The safest method is to use the glossary. This ensures that there is only a single place to maintain the information. It also ensures consistency if the same data is used in multiple use cases. It also makes the use case description cleaner and easier to read. The glossary can be used by your database and user interface designers more effectively.

While you should always strive to use the glossary, an alternative approach is to specify the detailed data requirements in the “Special Requirements” section of a use case or place them in the supplementary specification artifact.

Preconditions

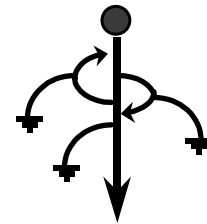
Preconditions

- ♦ Describes the state the system must be in before the use case can start.
 - Not the event that starts the use case.
- ♦ Reduce the amount of validation required.
- ♦ Optional: Use only if needed for clarification.

Get Quote

Precondition

The RU e-st system must have a connection to the Quote System in order to begin this use case.



33

IBM

A precondition for a use case is a condition in the system that must be true before the use case can begin. For example, the list of course offerings must be available before a student can register for courses. Otherwise, the student could not select from the course list.

A precondition is not the event starting the use case. For example, if a use case is written in such a way that logging on to the system is the first event in the use case, then logging on is not a precondition. But if a use case is written so that logging on must happen before the use case begins, then “The user has logged on to the system” would be a precondition.

Preconditions reduce the need for validation inside the use-case flows. In the example, you do not need to test that the course offerings are available anywhere in the use-case flows of events because, by definition, they must have been available or the use case could not execute.

Preconditions do not describe things outside the system. For example, “The customer has a valid PIN” would be an invalid precondition. Validating the PIN is something the system does in the use case.

Preconditions are optional. They are only used if they are needed for clarification.

Note: Preconditions can also be used at the beginning of an alternative flow or subflow. More commonly, they are used only on the basic flow. An example of using a precondition on an alternative flow is to describe security. If only certain actors can perform a given flow, you could attach a precondition to that flow to ensure the user has the appropriate authentication to perform the flow.

Postconditions

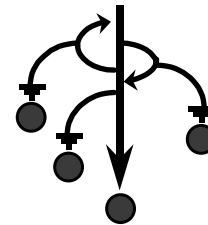
Postconditions

- ♦ Describes the state of the system at the end of the use case.
 - Guaranteed true when use case ends.
 - May contain variants.
- ♦ Optional: Use only if needed for clarification.

Execute Trade

Postcondition

At the end of this use case all accounts have been updated to reflect the transactions that have occurred.



34

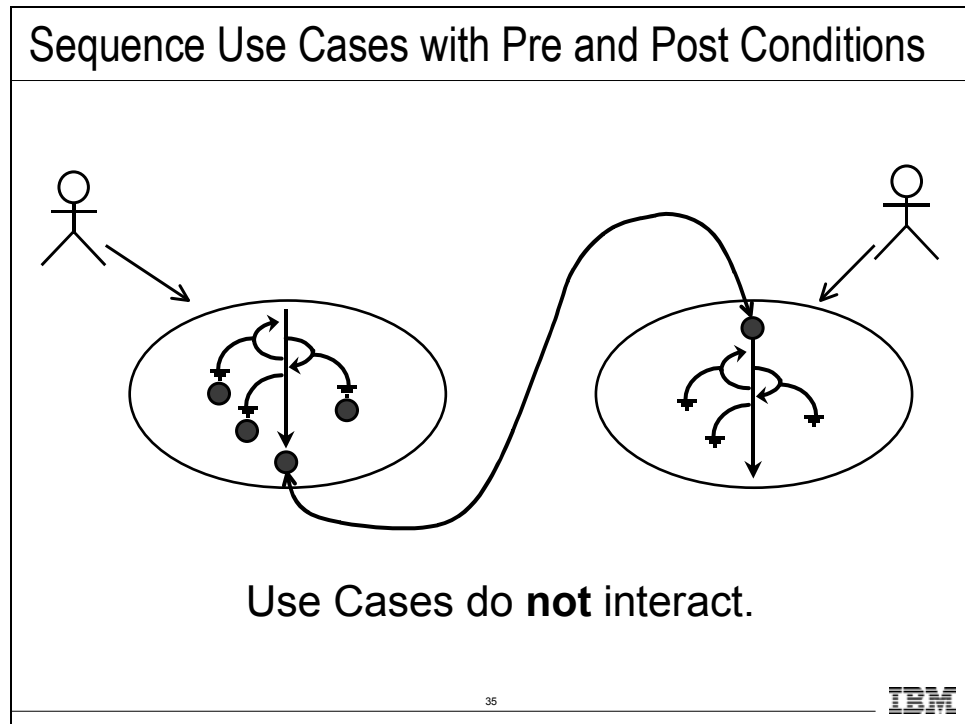
IBM

A postcondition is a condition that is guaranteed to be true after the use case ends. It must be true for a use case regardless of which alternative flows were executed. It should not be true only for the main flow. If something fails, you cover that in the postcondition by saying, "The action is completed," or "if something failed, the action is not performed," rather than just "The action is completed."

Postconditions can be a powerful tool for developing use cases. You must first define what the use case is supposed to achieve (the postcondition). Then describe the different ways to reach this condition (the flow of events needed).

Note: Postconditions can also be used at the end of alternative flows or subflows. But more commonly only one exists to describe the postcondition for the use case no matter how it ends.

Sequence Use Cases with Pre and Post Conditions



As described in the book *Use Case Modeling* by Kurt Bittner and Ian Spence, "Use cases do not directly communicate with one another. The only way for use cases to interact is via the state of the underlying system. Use cases can check the state of the system at any time, or wait for the state of the system to change, or can even be dependent on the state of the system via the use of preconditions."

For example, you may have a use case called "Manage Session," which handles authenticating a user when he/she logs on and then creates a session based on his/her user profile. When he/she logs off the system, the use case cancels the session. This use case starts with a precondition that there are no current active sessions in the system. When it authenticates the user, it sets the system state to "active session." The use case then waits until the user logs off. When the user logs off, it changes the system state to "no active session."

In order to ensure the user has logged onto the system before you start another use case, you would use a precondition of "There is an *active session* in the system."

(**Note:** *active session* should be in your glossary.) This has effectively ensured that the Manage Session use case has been run prior to the other use case.

Preconditions can only test the state of the system. For example, you cannot have a precondition that states, "**Use Case A** must have been executed before **Use Case B** can begin."

Note: Never draw an arrow between use cases as is shown in the example. The arrow in the slide illustrates the relationships between pre- and postconditions in two use cases.

Other Use Case Properties

Other Use Case Properties

- ◆ **Special requirements**
 - Related to this use case, not covered in flow of events.
 - Usually nonfunctional requirements, data, and business rules.
- ◆ **Extension points**
 - Name a set of places in the flow of events at which extending behavior can be inserted.
- ◆ **Relationships** (same as in Use-Case-Model Survey)
 - Associations with actors and other use cases.
- ◆ **Use-case diagrams**
 - Visual model of all relationships involving this use case.
- ◆ **Other diagrams or enclosures**
 - Interaction, activity, or other diagrams.

36



Special requirements are requirements relating to this use case that are not covered by the flow of events but that could influence the design of the system. These are usually nonfunctional constraints for the use case, such as reliability and performance.

Extension points name a set of places in the flow of events where extensions can be attached. Extensions and extension points are discussed in a later module.

Relationships list all the associations that the use case has to actors and to other use cases, with brief descriptions where applicable. You have already studied the “communicates-association” between an actor and a use case. Relationships between a use case and another use case is discussed in a later module.

Use-case diagrams: You may choose to illustrate how a use case relates to actors and other use cases in a use-case diagram (in unusual cases more than one diagram) owned by the use case. A diagram is useful if the use case is involved with many actors or has relationships to many other use cases. A diagram of this kind is of “local” character because it shows the use-case model from the perspective of one use case only.

Other diagrams and enclosures can be included if they help to clarify the use case. For example, interaction or activity diagrams can be used if they help to clarify the flow of events.

Use Case Tips

Use Case Tips

- ♦ Describe only the events visible to the actor:
 - What the actor does.
 - What the system does in response.
- ♦ Make use cases provide value to an actor.
- ♦ Use Cases may have differing levels of precision.
 - Detail until each stakeholder has a common understanding of the requirements from their perspective, then stop.
- ♦ Use agreed-upon terms and vocabulary.
- ♦ Use precise language.

37



Describe what the system does only if it is externally observable. The externally observable events are the software requirements or the things the system is required to do to satisfy user needs. In contrast, the things the system does that are not externally observable are really the responsibility of the system designers.

Each use case must provide a result of value to some actor. Each use case should be a complete flow of events until the actor's goal is achieved. If you have a use case that is too small, combine it with other use cases to make a complete flow that results in value to an actor.

For example, in a Course Registration System, a use case called "Enter Student ID" does not by itself result in value to the student actor. It should probably be made into a part of a larger use case, Register for Courses, which does result in value to the actor.

What level of detail should go into the use-case specification? Detail your flows until everyone has a common understanding of the requirements. Do not make the designers guess at details. The developers only know what the customer really wants **if** it is specified in the use case.

For a use case that all stakeholders have a thorough domain understanding, you will find that you might be able to get by with just an outline. For use cases that not all stakeholders have a good understanding of the domain, you need to have more detail in the use-case description. Do not detail use cases for the sake of it. You want to move onto analysis and design as quickly as possible, while ensuring everyone understands the requirements.

Unclear expressions may hide important information. Do not wait to understand the system. Be clear and accurate.

Use Case Checkpoints

Use Case Checkpoints

- ♦ Are the actor interactions and exchanged information clear?
- ♦ Does the communication sequence between actor and use case conform to the user's expectations?
- ♦ Is it clear how and when the use case's flow of events starts and ends?
- ♦ Is the subflow in a use case modeled accurately?



38

IBM

Exercise 8.2: Detail the Flows for Scenarios

Exercise 8.2: Detail the Flows for Scenarios

- ◆ Add detail to the step-by-step outline.
 - Basic flow
 - Alternative flows
- ◆ Identify preconditions.
- ◆ Identify postconditions.
- ◆ Revise Use-case scenario list.



39

IBM

See Student Workbook Exercise 8.2: Detail the Flows for Scenarios.

What About Nonfunctional Requirements?

What About Nonfunctional Requirements?

- ♦ The “URPS” of FURPS
 - Usability
 - Reliability
 - Performance
 - Supportability
- ♦ Compliance with Legal and Regulatory requirements
 - FCC
 - FDA
 - DOD
 - ISO
- ♦ Design Constraints
 - Operating Systems
 - Environments
 - Compatibility
 - Application Standards

Functionality	Feature set Capabilities	Generality Security
Usability	Human factors Aesthetics	Consistency Documentation
Reliability	Frequency/Severity of failure Recoverability	Predictability Accuracy MTBF
Performance	Speed Efficiency Resource usage	Throughput Response time
Supportability	Testability Extensibility Adaptability Maintainability Compatibility	Configurability Serviceability Installability Localizability Robustness

40



The URPS of FURPS specify the nonfunctional requirements of the system. They typically describe the quality aspects of the system.

What other types of nonfunctional requirements need to be specified?

Requirements must be verifiable. If you cannot verify that you have met the condition or capability, then it is not a requirement. In this case, it is better to specify as a constraint or goal and then create a requirement that can be tested. For example, how can you verify “the system must be available 99.9% of the time over a 12 month period?”

Do you run a test harness for a year? I hope not. Instead, specify it as a goal and then create a testable requirement that the customer agrees to. For example, “The system must be available, under load, 100 percent of the time over a one week period. During this time there must be no degradation of the following system resources: memory, available file handles, and disk space.”

Example: Nonfunctional Requirements

Example: Nonfunctional Requirements

- ◆ The RU e-st System
 - The system must provide price quotes that are no more than 15 minutes delayed.

- ◆ What are some others?
- ◆ Where should each of these be specified?

41



Two examples from the RU e-st stock trading system.

Specify Usability Requirements

Specify **Usability** Requirements

- ♦ Usability is the ease with which software can be learned and operated by the intended users.
- ♦ Usability requirements include:
 - Training time requirements, measurable task times.
 - User abilities (beginner/advanced).
 - Comparison to other systems that users know and like.
 - Online help systems, tool tips, documentation needs.
 - Conformity with standards.
 - Examples: Windows, style guides, GUI Standards

*All user documentation must conform to the
Microsoft® Manual of Style for Technical Publications.*

42



How can you “quantify” usability so it can be testable?

The ISO9241 specification provides a similar definition of usability: *The effectiveness, efficiency and satisfaction with which specified users achieve specified goals in particular environments.*

Specify Reliability Requirements

Specify **Reliability** Requirements

- ♦ Reliability is the the ability for the software to behave consistently in a user-acceptable manner.
- ♦ Reliability requirements include:
 - Availability (xx.xx%)
 - Accuracy
 - MTBF (xx hrs)
 - Max. bugs per/KLOC (0-x)

Reporting of velocity for the Mars Lander must be in units of meters per second and accurate to 1 in 1e6.

Davis Workshop, 1993

43



How can we quantify reliability so that it is testable?

MTBF – Mean Time Between Failures

KLOC – Thousand Lines of Code

The example given is an accuracy requirement. The subsystems in the space probe need reliable/accurate measurements in order to ensure that the probe will not plummet into the surface of the planet or miss the planet entirely.

Specify Performance Requirements

Specify **Performance** Requirements

- ♦ Performance is a measure of speed or efficiency of the running system.
- ♦ Performance requirements include:
 - Capacity
 - Throughput
 - Response time
 - Memory
 - Degradation modes
 - Use of scarce resources
 - Processor, memory, disk, network bandwidth

There must be no more than four protocol exchanges, of no greater size than 512k bytes each, between the client and the server for each user action.

Davis Workshop, 1993

44



How can you “quantify” performance so that it is testable?

The example is based on a *Network Sociability Guidelines* standard that is used in an Australian telecommunications company. The requirement is in place to ensure that any new application that is brought on line has a predictable impact on the corporate WAN. “Each user action” means any keyboard or mouse action. So, for example, if the user clicks an OK button, the client and server are allowed no more than four protocol exchanges. Each exchange is limited to a maximum of 512 kilobytes of data.

Specify Supportability Requirements

Specify **Supportability** Requirements

- ♦ Supportability is the ability of the software to be easily modified to accommodate enhancements and repairs.
- ♦ Supportability requirements include:
 - Languages, DBMS, tools, and so on.
 - Programming standards.
 - Error handling and reporting standards.
- ♦ Often difficult to specify
 - If not measurable or observable, it is not a requirement.
 - Is it a design constraint?
 - Is it an intent or goal?

Davis Workshop, 1993

45



How can you “quantify” supportability so that it is testable? Here are examples of observable standards and tools that may be specified to enhance supportability.

The supportability area is the hardest to specify as observable or measurable requirements. Often these items are stated as design constraints, or, they may be stated as intent and goals in the Vision document.

An example of a supportability requirement is: “The Web system must remain available to users while it is being upgraded.” This requirement would have a significant architectural impact on the system.

How to Describe Communication Protocols

How to Describe Communication Protocols

- ◆ Specify a communication protocol if the actor is another system or external hardware.
 - The description of the use case should state if some existing protocol is to be used.
 - If the protocol is new, it will be fully described during object-model development.

46



You might need to define a communication protocol if the actor is another system or external hardware. The description of the use case should state if some existing protocol (http, tcp/ip) is to be used. If the protocol is new, you must fully describe it during the object-model development.

Exercise 8.3: Identify Nonfunctional Requirements

Exercise 8.3: Identify Nonfunctional Requirements

- ◆ Identify nonfunctional requirements.
- ◆ Determine where nonfunctional requirements should be specified.



47

IBM

See Student Workbook Exercise 8.3: Identify Nonfunctional Requirements.

Review: Refine the System Definition

Review: Refine the System Definition

1. When are use cases used to specify requirements?
2. When are declarative statements used?
3. What is the difference between black and white-box use cases? Which is better? Why?
4. What are some alternatives for expressing conditional behavior? What are the benefits and drawbacks of each alternative?
5. When would you use a subflow?
6. How should you handle describing forms or data types in a use case?
7. What are the benefits of using pre- and postconditions?

48



- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.